Technical Design Document (TDD)

SECTION 1 — SYSTEM IMPLEMENTATION ARCHITECTURE

1.1 Purpose of This Document

This document specifies:

- Exact technology stack
- Concrete implementation details
- Service boundaries
- Database schema structure
- API contract format
- Event dispatcher mechanics
- ML integration plan
- Deployment architecture
- Operational constraints

This document is intended for:

- Developers
- DevOps engineers
- ML engineers
- Reviewers validating implementation feasibility

This is not conceptual.
This is executable design.

1.2 Finalized Technology Stack

Backend

- Language: Python 3.11
- Framework: FastAPI
- ASGI Server: Uvicorn (with Gunicorn for production)
- ORM: SQLAlchemy 2.0

- Migration Tool: Alembic
- Data Validation: Pydantic v2
- Auth: JWT (python-jose)

Database

- Engine: PostgreSQL 15
- Hosting: GCP Cloud SQL
- Extensions:
    - uuid-ossp
    - pgcrypto
    - pg_stat_statements (monitoring)
- Data types used:
    - UUID
    - JSONB
    - TIMESTAMP WITH TIME ZONE

Event System

- In-process dispatcher
- DB-backed event persistence
- Partition key: crop_instance_id
- Row-level locking for isolation
- Idempotency enforced via unique event_hash

Future-ready abstraction for Pub/Sub migration.

ML Stack

- Risk Model: Scikit-learn (LogisticRegression)
- CNN (backend): PyTorch
- Edge Model: TensorFlow Lite
- Feature Extraction: OpenCV + NumPy
- Model serialization: joblib / torch.save

Media

- Storage: GCP Cloud Storage (Private bucket)

- Access: Signed URLs (15 min expiry)
- Processing: Separate Cloud Run worker service
- Video Frame Extraction: OpenCV

Frontend

- Framework: Next.js (React 18)
- Styling: TailwindCSS
- State Management: React Context + minimal Redux (if needed)
- PWA: Service Worker enabled
- Auth storage: HttpOnly cookie (recommended)

Deployment

- Backend API: Cloud Run service
- Media Worker: Cloud Run service
- Video Worker: Cloud Run service (optional)
- Database: Cloud SQL
- Storage: Cloud Storage
- Cron Jobs: Cloud Scheduler
- Logging: Cloud Logging
- Monitoring: Cloud Monitoring

1.3 High-Level Service Topology

```
[ Next.js Frontend ]
      ↓ HTTPS
[ Cloud Run - Backend API ]
      ↓
[ Cloud SQL - PostgreSQL ]

[ Cloud Run - Media Worker ]
      ↓
[ Cloud Storage ]

[ Cloud Scheduler ]
      ↓
[ Backend API / Worker endpoints ]
```

No service shares in-memory state.

All state stored in DB.

1.4 Service Boundaries

Backend API Service Responsibilities

- Authentication
- Crop lifecycle operations
- Action logging
- Replay engine
- Risk computation
- SOE management
- Admin governance
- Event dispatch processing
- ML inference (lightweight)
- Media upload orchestration

Must NOT:

- Perform heavy video frame processing
- Store files locally
- Hold long-lived in-memory state

Media Worker Responsibilities

- Image inference
- Video frame extraction
- CNN inference
- Feature aggregation
- Emit MediaAnalyzed event

Must NOT:

- Modify CTIS state directly
- Bypass event dispatcher

Database Responsibility

Single source of truth for:

- CropInstances
- Action logs
- Event logs
- Trust scores
- ML audit
- Snapshots
- Deviation profiles

1.5 Directory Structure (Backend)

```
cultivax/
│
├── app/
│   ├── main.py
│   ├── config.py
│   ├── database.py
│   ├── models/
│   ├── schemas/
│   ├── api/
│   ├── services/
│   │   ├── ctis/
│   │   ├── soe/
│   │   ├── event_dispatcher/
│   │   ├── ml/
│   │   ├── media/
│   │   ├── replay/
│   ├── workers/
│   ├── utils/
│   └── security/
│
├── alembic/
├── Dockerfile
├── requirements.txt
└── .env
```

Clear modular boundaries.

1.6 Core Runtime Flow (Implementation-Level)

Example: Log Action

1. API endpoint receives request
2. Validate via Pydantic schema
3. Verify JWT
4. Insert ActionLog (pending)
5. Create EventLog entry (ActionLogged)
6. Commit transaction
7. Dispatcher processes event
8. ReplayTriggered event created
9. Replay executes
10. Snapshot updated
11. Response returned

All DB writes wrapped in transaction.

1.7 Concurrency & Locking Strategy

For replay:

- Acquire SELECT FOR UPDATE on crop_instance row
- Prevent concurrent replay
- Queue other events
- Release lock after commit

Prevents race conditions.

1.8 Migration Readiness Constraints

To allow migration to Pub/Sub later:

- Dispatcher interface abstracted
- Event payload serializable
- No business logic inside dispatcher
- Events immutable

This ensures scaling to 500–600 users requires no rewrite.

1.9 Non-Functional Guarantees

- Replay O(n) with snapshot optimization
- Media inference async
- API latency target < 300ms (excluding media)
- Event processing idempotent
- DB ACID consistency enforced
- No mutation without event

📘 SECTION 2 — DATABASE SCHEMA DESIGN

2.1 Database Design Principles

All mutable state must be event-driven.

No direct state mutation without event_log entry.

Replay must be reconstructable from action_logs.

All destructive operations are soft-delete.

Idempotency enforced at DB level.

All tables must support migration without rewrite.

No in-memory-only data.

2.2 Core Identity Tables

2.2.1 users

```
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    full_name TEXT NOT NULL,
    phone_number TEXT UNIQUE NOT NULL,
    role TEXT NOT NULL CHECK (role IN ('farmer','provider','admin')),
    password_hash TEXT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);
```

Indexes:

```
CREATE INDEX idx_users_role ON users(role);
```

2.3 CTIS Core Tables

2.3.1 crop_instances

```
CREATE TABLE crop_instances (
    crop_instance_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id),
    crop_type TEXT NOT NULL,
    region TEXT NOT NULL,
    sowing_date DATE NOT NULL,
    rule_version_applied INT NOT NULL,
    baseline_stage TEXT NOT NULL,
    current_stage TEXT NOT NULL,
    stress_score NUMERIC(5,4) DEFAULT 0.0,
    risk_index NUMERIC(5,4) DEFAULT 0.0,
    state TEXT CHECK (state IN ('Active','AtRisk','Harvested','RecoveryRequired')),
    event_chain_hash TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);
```

Indexes:

```
CREATE INDEX idx_crop_user ON crop_instances(user_id);
CREATE INDEX idx_crop_state ON crop_instances(state);
```

2.3.2 action_logs

```
CREATE TABLE action_logs (
    action_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    crop_instance_id UUID REFERENCES crop_instances(crop_instance_id),
    action_type TEXT NOT NULL,
    action_impact_type TEXT NOT NULL,
    action_effective_date DATE NOT NULL,
    local_monotonic_counter BIGINT NOT NULL,
    metadata JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    deleted_at TIMESTAMPTZ,
```

```
    UNIQUE (crop_instance_id, action_effective_date, local_monotonic_counter)
);
```

Replay index:

```
CREATE INDEX idx_action_replay
ON action_logs(crop_instance_id, action_effective_date DESC, local_monotonic_counter DESC);
```

2.3.3 crop_instance_snapshots

```
CREATE TABLE crop_instance_snapshots (
    snapshot_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    crop_instance_id UUID REFERENCES crop_instances(crop_instance_id),
    snapshot_index INT NOT NULL,
    stage TEXT NOT NULL,
    baseline_stage TEXT NOT NULL,
    stress_score NUMERIC(5,4),
    risk_index NUMERIC(5,4),
    deviation_profile JSONB,
    rule_version_applied INT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Index:

```
CREATE INDEX idx_snapshot_lookup
ON crop_instance_snapshots(crop_instance_id, snapshot_index DESC);
```

2.3.4 deviation_profiles

```
CREATE TABLE deviation_profiles (
    profile_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    crop_instance_id UUID REFERENCES crop_instances(crop_instance_id),
    consecutive_deviation_count INT DEFAULT 0,
    deviation_trend_slope NUMERIC(6,4),
    recurring_pattern_flag BOOLEAN DEFAULT FALSE,
    last_updated TIMESTAMPTZ DEFAULT NOW()
);
```

2.3.5 yield_records

```
CREATE TABLE yield_records (
```

```
    yield_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    crop_instance_id UUID REFERENCES crop_instances(crop_instance_id),
    reported_yield NUMERIC(10,2) NOT NULL,
    ml_yield_value NUMERIC(10,2),
    yield_verification_score NUMERIC(5,4),
    submitted_at TIMESTAMPTZ DEFAULT NOW()
);
```

2.4 Event System Tables

2.4.1 event_log

```
CREATE TABLE event_log (
    event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    crop_instance_id UUID REFERENCES crop_instances(crop_instance_id),
    event_type TEXT NOT NULL,
    event_payload JSONB NOT NULL,
    event_hash TEXT UNIQUE NOT NULL,
    chain_hash TEXT,
    status TEXT CHECK (status IN ('Created','Processing','Processed','Failed','DeadLetter')),
    retry_count INT DEFAULT 0,
    failure_reason TEXT,
    priority INT DEFAULT 5,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    processed_at TIMESTAMPTZ
);
```

Indexes:

```
CREATE INDEX idx_event_partition
ON event_log(crop_instance_id, status, priority, created_at);
```

2.5 SOE Tables

2.5.1 service_providers

```
CREATE TABLE service_providers (
    provider_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id),
    equipment_type TEXT,
```

```
    trust_score NUMERIC(5,4) DEFAULT 0.5,
    completion_rate NUMERIC(5,4),
    complaint_ratio NUMERIC(5,4),
    is_verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);
```

2.5.2 service_requests

```
CREATE TABLE service_requests (
    request_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    crop_instance_id UUID REFERENCES crop_instances(crop_instance_id),
    provider_id UUID REFERENCES service_providers(provider_id),
    status TEXT CHECK (status IN ('Pending','Accepted','Completed','Cancelled','Escalated')),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

2.5.3 service_reviews

```
CREATE TABLE service_reviews (
    review_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    request_id UUID REFERENCES service_requests(request_id),
    rating INT CHECK (rating BETWEEN 1 AND 5),
    complaint_category TEXT,
    severity_level INT,
    resolution_status TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

2.6 ML Tables

2.6.1 ml_models

```
CREATE TABLE ml_models (
    model_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_name TEXT NOT NULL,
    model_version TEXT NOT NULL,
```

```
    model_type TEXT,
    metrics JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

2.6.2 ml_training_audit

```
CREATE TABLE ml_training_audit (
    training_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_version TEXT,
    dataset_reference TEXT,
    feature_schema_version TEXT,
    evaluation_metrics JSONB,
    training_start TIMESTAMPTZ,
    training_end TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

2.7 Media Tables

2.7.1 media_files

```
CREATE TABLE media_files (
    media_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    crop_instance_id UUID REFERENCES crop_instances(crop_instance_id),
    file_type TEXT CHECK (file_type IN ('image','video')),
    storage_path TEXT NOT NULL,
    confidence_score NUMERIC(5,4),
    image_quality_score NUMERIC(5,4),
    extracted_features JSONB,
    source TEXT CHECK (source IN ('backend','edge')),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    scheduled_deletion_at TIMESTAMPTZ
);
```

Index:

```
CREATE INDEX idx_media_crop
ON media_files(crop_instance_id);
```

2.8 Admin & Audit

2.8.1 admin_audit_log

```
CREATE TABLE admin_audit_log (
    audit_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    admin_id UUID REFERENCES users(user_id),
    action_type TEXT,
    target_id UUID,
    previous_state JSONB,
    new_state JSONB,
    reason TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Immutable rule:

No DELETE allowed.

Only insert.

2.9 Retention Policy Implementation

Scheduled job:

Delete media_files where scheduled_deletion_at < NOW()

Keep extracted_features

Log deletion event

2.10 Migration Readiness

All tables use UUID PK

JSONB flexible for schema evolution

Index strategy supports scaling

No hard constraints blocking future partitioning

Can partition action_logs by crop_instance_id later

📘 SECTION 3 — EVENT DISPATCHER IMPLEMENTATION DESIGN

3.1 Design Goals

The Event Dispatcher must:

1. Persist every event before execution.
2. Enforce idempotency at DB level.
3. Process events in FIFO order per crop_instance.
4. Isolate partitions by crop_instance_id.
5. Guarantee atomic replay.
6. Support retry and dead-letter handling.
7. Be easily replaceable with Pub/Sub in future.
8. Avoid global in-memory state.

3.2 Dispatcher Architecture Overview

Conservative implementation:

- In-process dispatcher inside FastAPI app
- Uses DB event_log table as source of truth
- Uses SELECT FOR UPDATE SKIP LOCKED
- Processes events in small batches
- Partitioned logically by crop_instance_id

There is NO in-memory queue that stores unpersisted state.

3.3 Core Abstraction Layer (Migration-Ready)

We define a dispatcher interface.

```
class EventDispatcherInterface:
    async def publish(self, event_type: str, payload: dict, crop_instance_id: UUID):
        pass

    async def process_pending_events(self):
        pass
```

Current implementation:
DBEventDispatcher

Future implementation:
PubSubEventDispatcher

Business logic never directly depends on concrete implementation.

3.4 Event Lifecycle Model (Concrete)

Event states in DB:

- Created
- Processing
- Processed
- Failed
- DeadLetter

Lifecycle:

1. publish()
    a. Insert event_log row
    b. status = 'Created'
    c. event_hash generated
2. process_pending_events()
    a. Fetch events WHERE status='Created'
    b. Order by priority, created_at
    c. Lock rows
    d. status='Processing'
    e. Execute handler
    f. On success → status='Processed'
    g. On failure → increment retry_count
        i. If retry_count < threshold → status='Created'
        ii. Else → status='DeadLetter'

3.5 Event Hash & Idempotency Enforcement

When publishing:

```
event_hash = sha256(
    f"{event_type}:{crop_instance_id}:{json.dumps(payload)}"
)
```

DB has:

UNIQUE(event_hash)

If duplicate publish attempt occurs:

- Insert fails
- Existing event returned

- No duplicate processing

This prevents:

- Double action logging
- Network retry duplication
- Offline sync duplication

3.6 Partitioned Processing Strategy

Partition key:
crop_instance_id

Processing query:

```
SELECT *
FROM event_log
WHERE status = 'Created'
ORDER BY priority DESC, created_at ASC
LIMIT 20
FOR UPDATE SKIP LOCKED;
```

Then:

Before processing each event:

```
SELECT * FROM crop_instances
WHERE crop_instance_id = ?
FOR UPDATE;
```

This ensures:

- Only one replay per crop at a time
- No cross-crop blocking
- No deadlock cascade

3.7 Replay Trigger Mechanism

Certain events automatically emit ReplayTriggered:

Events that require replay:

- ActionLogged
- YieldSubmitted

- SowingDateModified
- MediaAnalyzed (if stress updated)

Replay event:

event_type = 'ReplayTriggered'

Replay logic:

- Acquire crop lock
- Load snapshot (if exists)
- Load action_logs
- Recompute state
- Update crop_instances row
- Insert snapshot if threshold met
- Commit transaction

Replay is atomic.

If replay fails:

- Rollback transaction
- Mark event Failed
- Do not partially update crop_instance

3.8 Dead Letter Queue Handling

If retry_count exceeds threshold (e.g., 5):

- status='DeadLetter'
- Insert admin alert
- Mark crop_instance state='RecoveryRequired'

DeadLetter events do NOT block other events.

Admin endpoint:

GET /api/v1/admin/dead-letters

Allows inspection and manual retry.

3.9 Backpressure Strategy

If event backlog grows:

- process_pending_events() runs in background loop
- Max concurrent replays limited (e.g., 5)
- Use asyncio.Semaphore to control concurrency

If queue length > threshold:

- Raise monitoring alert
- Reduce intake rate (soft throttle)

Prevents CPU exhaustion.

3.10 Debounce Replay Optimization

If multiple replay-triggering events arrive quickly:

- Merge them logically
- Only one ReplayTriggered inserted per crop per short interval (e.g., 300ms window)

Prevents replay storm during bulk sync.

3.11 Crash Recovery Behavior

On service restart:

- Any event with status='Processing'
  → Reset to 'Created'
  → retry_count += 1

Ensures:

- No event permanently stuck
- No partial state applied

Because state changes occur only after successful commit.

3.12 Event Dependency Map (Hard-Coded Rules)

Allowed chains:

ActionLogged → ReplayTriggered
YieldSubmitted → ReplayTriggered

MediaAnalyzed → ReplayTriggered
RuleModified → (No automatic replay)

Forbidden:

RegionalClusterUpdated → ReplayTriggered (retroactively)

This prevents infinite loops.

3.13 Concurrency Model

Event loop:

- Background task started in FastAPI startup event
- Async loop polling DB every X ms
- Uses SKIP LOCKED to avoid contention
- Each Cloud Run instance runs its own loop

Row-level locking ensures no double execution across instances.

3.14 Performance Envelope

Expected load:

- 200 users
- ~20 concurrent sessions
- <100 events/minute

This design handles that easily.

For 600 users:

- Increase Cloud Run max instances
- Increase DB pool size
- Possibly reduce batch size
- Still no rewrite required

3.15 Migration Plan to Pub/Sub

Because:

- EventDispatcherInterface exists
- Event payload serializable
- No business logic inside dispatcher

Migration steps:

1. Replace DBEventDispatcher.publish()
2. Publish to Pub/Sub topic
3. Subscriber service consumes events
4. Same handler functions reused

Zero business logic rewrite.

3.16 Security Considerations

- No direct table mutation outside event processing
- All event handlers wrapped in transaction
- Replay cannot be triggered without event entry
- Event hash prevents tampering
- Chain hash verification can be added

📘 SECTION 4 — CTIS IMPLEMENTATION DESIGN

4.1 CTIS Core Responsibilities

CTIS must:

1. Maintain chronological integrity.
2. Recompute crop state deterministically.
3. Enforce invariants.
4. Apply drift constraints.
5. Integrate multi-signal stress safely.
6. Separate Farmer Truth from ML Truth.
7. Never mutate without event trigger.
8. Support snapshot optimization.

4.2 Core Data Inputs for Replay

Replay uses:

- crop_instances
- action_logs (ordered)
- rule_version_applied
- deviation_profiles
- yield_records
- latest stress signals
- weather risk (current snapshot)

Replay does NOT:

- Depend on external API live calls
- Use ephemeral in-memory values

All needed data must be stored or versioned.

4.3 Chronological Invariants (Code-Level Enforcement)

Before replay begins:

```
def validate_invariants(crop, actions):
    for action in actions:
        assert action.action_effective_date >= crop.sowing_date
        assert action.local_monotonic_counter >= 0
```

Additionally:

- No action date beyond current_date + tolerance_limit
- No stage skip allowed
- No negative stress
- No direct state mutation without event

If invariant fails:

- Raise ReplayIntegrityException
- Mark crop_instance as RecoveryRequired

4.4 Replay Algorithm (Precise Pseudocode)

```
def replay_crop_instance(crop_instance_id):

    acquire_row_lock(crop_instance_id)

    crop = load_crop_instance()
    actions = load_ordered_actions()

    snapshot = load_latest_snapshot()

    if snapshot exists:
        state = snapshot_state
        start_index = snapshot.snapshot_index
    else:
        state = initialize_from_baseline()
        start_index = 0

    for action in actions[start_index:]:
        validate_action_invariant(action)
        state = apply_action(state, action)

    state = apply_stress_integration(state)
    state = compute_risk_index(state)
    state = enforce_drift_constraints(state)

    update_crop_instance(state)
    maybe_create_snapshot()

    release_lock()
```

Atomic transaction enforced.

4.5 Stage Transition Logic

Each crop has:

CropRuleTemplate:

```
{
  "stages": [
    { "name": "Germination", "duration_days": 7 },
    { "name": "Tillering", "duration_days": 21 },
    ...
```

```
  ]
}
```

Baseline stage computed as:

```
days_since_sowing = today - sowing_date
baseline_stage = stage_lookup(days_since_sowing)
```

Actual stage adjusted by:

- deviation_offset
- stress_modifier

But must not exceed:

- max_stage_drift (e.g., ±14 days)

No direct stage jump allowed.

4.6 Drift Constraint Enforcement

After state computation:

```
if abs(state.stage_offset) > crop.max_allowed_drift:
   state.stage_offset = clamp(state.stage_offset)
```

Prevents:

- Infinite delay
- Infinite acceleration

Drift bounded per crop type.

4.7 Stress Integration Formula

Multi-signal arbitration:

```
stress_signal = (
   w_backend * backend_ml_stress +
   w_weather * weather_risk +
   w_deviation * deviation_penalty +
   w_edge * edge_signal
)

new_stress = alpha * (stress_signal * confidence) + (1 - alpha) * previous_stress
```

Where:

w_backend > w_weather > w_deviation > w_edge

Constraints:

- stress in [0,1]
- max daily increase limit
- max single-event jump limit

4.8 Risk Index Computation

System Risk Index:

risk_index = (weather_risk * 0.7) + (farmer_input_risk * 0.3)

Adjusted by:

- stress_score
- stage sensitivity factor

Must remain probabilistic (0–1).

4.9 Yield Truth Enforcement

When YieldSubmitted event processed:

```
reported_yield = input_value

if reported_yield > biological_limit:
   ml_yield_value = biological_limit
else:
   ml_yield_value = reported_yield
```

Store both.

UI always shows reported_yield.

ML training uses ml_yield_value.

Never modify reported_yield.

4.10 Snapshot Strategy

Snapshot created when:

- action_count % 50 == 0
  OR
- major structural action occurs

Snapshot stores:

- stage
- stress_score
- risk_index
- deviation_profile

Reduces replay cost.

4.11 Structural Action Handling

If action_impact_type == 'Structural':

- Invalidate snapshot
- Full replay required
- Recompute baseline

Examples:

- SowingDateModified

4.12 Behavioral Adaptation Enforcement

Deviation logic:

- Track consecutive deviation count
- If recurring pattern detected:
  - Apply small bounded adaptation offset
  - Must not exceed max_adaptation_days

Reset at season end.

4.13 What-If Simulation Implementation

For What-If:

- Clone crop_instance state in memory
- Apply hypothetical action
- Run replay logic
- Do NOT persist
- Return projected state

Never emit event.

4.14 State Transition Safeguards

Valid transitions:

Active → AtRisk
Active → Harvested
AtRisk → Active
Active → RecoveryRequired

Invalid transitions blocked by:

```
if not is_valid_transition(old_state, new_state):
    raise InvalidStateTransition()
```

4.15 Concurrency Safety

Replay executed inside:

- DB transaction
- Row-level lock

No two replays on same crop simultaneously.

4.16 Performance Envelope

With snapshots:

Replay complexity:

O(k) where k = actions since last snapshot.

For 200 users:

Negligible overhead.

For 600 users:

Still safe with tuning.

4.17 ML Kill Switch Integration

If feature_flag.ml_enabled == False:

- Skip ML signal in stress integration
- Use rule-only computation

No code branching outside controlled function.

4.18 Logging & Observability

Each replay logs:

- replay_duration
- actions_processed
- snapshot_used
- drift_applied
- stress_delta

Helps debugging and performance tuning.

📘 SECTION 5 — SERVICE ORCHESTRATION ENGINE (SOE) IMPLEMENTATION DESIGN

5.1 SOE Responsibilities

SOE handles:

1. Service provider registration
2. Equipment availability
3. Service request matchmaking
4. Complaint handling
5. Trust score computation
6. Exposure fairness
7. Review audit
8. Escalation workflow

SOE must NOT:

- Mutate crop stage
- Trigger replay
- Influence CTIS baseline
- Override farmer authority

CTIS and SOE are logically separate systems.

5.2 Provider Registration Flow

When provider registers:

1. Create user (role='provider')
2. Insert into service_providers
3. Default trust_score = 0.5
4. is_verified = False

Admin may later verify provider.

No trust score advantage given for early signup.

5.3 Equipment & Availability Model

Add availability table:

```
CREATE TABLE provider_availability (
  availability_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  provider_id UUID REFERENCES service_providers(provider_id),
  available_from DATE,
```

```
    available_to DATE,
    equipment_status TEXT CHECK (equipment_status IN ('Available','Maintenance','Unavailable'))
);
```

Match requests only if:

```
current_date ∈ available window
equipment_status = 'Available'
```

5.4 Service Request Flow (Implementation)

Service request lifecycle:

1. Farmer creates request
2. Matching providers retrieved
3. Sorted by exposure-adjusted trust
4. Provider accepts or declines
5. Completion status updated
6. Review submitted
7. Trust recalculated

All mutations wrapped in transaction.

5.5 Trust Score Model (Mathematically Defined)

Trust score components:

- Completion Rate (CR)
- Complaint Ratio (CPR)
- Average Rating (AR)
- Verified Bonus (VB)
- Escalation Penalty (EP)

Formula:

```
trust_score = (
    w1 * CR +
    w2 * (1 - CPR) +
    w3 * normalized_rating +
    w4 * VB -
    w5 * EP
)
```

Weights:

w1 > w2 > w3
w4 small bonus
w5 moderate penalty

All values normalized 0–1.

Clamp result to [0,1].

5.6 Trust Decay Mechanism

To prevent permanent dominance:

Apply time decay:

decay_factor = exp(-lambda * days_since_last_completion)
trust_score *= decay_factor

Ensures inactive providers gradually lose ranking advantage.

5.7 Exposure Fairness Algorithm

When listing providers:

Sort by:

ranking_score = trust_score - exposure_penalty

Exposure penalty computed as:

exposure_penalty = alpha * recent_impressions_count

This prevents:

- Top providers monopolizing visibility
- New providers being invisible

Ensures marketplace fairness.

5.8 Complaint Handling Logic

Complaint submission:

- Insert service_review
- Update complaint_ratio
- If severity > threshold:
    - Escalate to admin
    - status = 'Escalated'

Admin resolution required for:

- Repeated high-severity complaints
- Fraud suspicion

Escalation does NOT block other provider operations automatically.

5.9 Fraud Detection Rules

Flag if:

- Same IP multiple reviews
- Review from linked account
- Sudden rating spike
- High complaint but high rating mismatch

Flagged providers:

- Mark under review
- Reduce exposure temporarily

No automatic ban without admin review.

5.10 CTIS Isolation Guarantee

SOE must never:

- Call replay
- Modify crop_instance
- Influence stress_score
- Influence risk_index

Only shared link:

service_requests references crop_instance_id.

That is informational only.

5.11 Concurrency Model

For trust recalculation:

- Lock provider row FOR UPDATE
- Recalculate
- Update atomically

Prevents concurrent inconsistent trust updates.

5.12 Service Review Constraints

Rules:

- Only one review per request
- Cannot review before completion
- Cannot self-review

DB-level constraints:

```
ALTER TABLE service_reviews
ADD CONSTRAINT unique_review_per_request UNIQUE(request_id);
```

5.13 Escalation Workflow

Escalation path:

Complaint → Severity Check → Escalate → Admin Review → Resolution

Admin options:

- Dismiss
- Warn provider
- Reduce trust score
- Suspend provider (soft delete)
- Permanently deactivate

All admin actions logged in admin_audit_log.

5.14 Performance Envelope

Expected load:

- <50 service requests per day
- Trust recalculation minimal CPU
- Exposure sorting lightweight

Safe up to 600 users.

Beyond 1000:

- Add caching layer
- Precompute ranking score

No redesign needed.

5.15 Migration Readiness

If SOE needs to scale independently:

- Extract into separate microservice
- Keep DB shared initially
- Later move to separate DB

Because:

- Logic is self-contained
- No CTIS dependency

5.16 Observability

Log:

- Trust recalculation
- Escalation events
- Exposure adjustments
- Fraud flags

Helps audit marketplace fairness.

📘 SECTION 6 — ML IMPLEMENTATION DESIGN

6.1 ML Design Principles

1. ML must never override deterministic CTIS rules.
2. ML outputs must be bounded and probabilistic.
3. All ML outputs must carry confidence.
4. All training runs must be auditable.
5. Small dataset bias must be controlled.
6. No PII used in training.
7. All model versions stored and traceable.

6.2 ML System Architecture

ML Components:

1. Risk Prediction Model (Logistic Regression)
2. Stress Detection CNN (PyTorch)
3. Regional Cluster Learning (Statistical)
4. Yield Verification Model
5. Confidence Propagation Layer

ML runs inside backend container
Heavy training runs as background job.

6.3 Feature Engineering Pipeline

6.3.1 Risk Model Features

Input features:

- stress_score
- deviation_count
- deviation_trend_slope

- weather_risk
- days_since_sowing
- action_density_last_14_days
- historical_average_yield_region
- crop_stage_numeric

All features normalized to [0,1] or standardized.

Feature schema version tracked:

feature_schema_version = "v1.0"

Stored in ml_training_audit.

6.4 Risk Model (Logistic Regression)

Purpose:

Predict probability of AtRisk state.

Implementation:

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Output:

```
risk_probability = model.predict_proba(X)[1]
```

Constraints:

- Output ∈ [0,1]
- Must not exceed CTIS stage drift authority
- Only influences risk_index, not stage directly

If ML disabled:

Use rule-based fallback.

6.5 Stress Detection CNN (Backend)

Model type:

Lightweight CNN (e.g., ResNet18 variant)

Input:

- 224x224 image
- RGB normalized

Output:

- stress_probability ∈ [0,1]
- confidence_score ∈ [0,1]

Inference pipeline:

image → preprocessing → model.forward() → sigmoid

Post-processing:

stress_weighted = stress_probability * confidence_score

Bounded daily stress increase enforced.

6.6 Multi-Angle Media Aggregation

When multiple images uploaded:

Aggregate:

```
aggregate_stress = weighted_mean(
   stress_i,
   weight = confidence_i
)
```

If variance high:

Reduce final confidence.

Prevents single bad frame corruption.

6.7 Video Frame Pipeline

Video worker:

1. Extract 1 frame per N seconds
2. Run CNN inference per frame
3. Remove low-quality frames
4. Aggregate via trimmed mean

5. Compute final stress + confidence

Heavy processing isolated in worker.

6.8 Regional Cluster Learning

Purpose:

Detect regional adaptation patterns.

Method:

- Group by region + crop_type
- Collect verified yield + deviation patterns
- Use K-Means clustering OR statistical mean shift

Constraint:

Minimum sample size threshold (e.g., 30 instances)

If cluster_size < threshold:

Do NOT apply adaptation.

Cluster update rule:

Prospective only
No retroactive mutation.

6.9 Yield Verification Logic

When farmer submits yield:

Compute:

z_score = (reported_yield - regional_mean) / regional_std

If |z_score| > threshold:

- reduce yield_verification_score
- clamp ML yield value

Never modify reported_yield.

Store:

reported_yield
ml_yield_value
yield_verification_score

6.10 Confidence Propagation Layer

Every ML output must include:

confidence_score

Stress integration uses:

final_stress_delta = raw_stress * confidence_score

Low confidence reduces impact.

Confidence sources:

- Image quality score
- Frame consistency
- Model certainty
- Cluster stability
- Sample size adequacy

6.11 Small Dataset Safeguards

Because initial dataset small:

Rules:

- Freeze ML if dataset < minimum threshold
- Use rule-based risk only
- Apply stronger smoothing factor
- Reduce stress weight

This prevents overfitting early.

6.12 Model Versioning Strategy

Each model:

- Stored with model_version
- Version saved in crop_instance
- Inference logs include model_version
- Training runs logged in ml_training_audit

Allows rollback if needed.

6.13 Training Lifecycle

Training triggered:

- Manually by admin
- Or scheduled batch (Cloud Scheduler)

Training pipeline:

1. Extract dataset
2. Filter invalid entries
3. Remove outliers
4. Split train/test
5. Evaluate metrics
6. Save model
7. Log metrics
8. Activate model

No automatic activation without admin approval.

6.14 ML Kill Switch

Feature flag:

ml_enabled = True/False

If False:

- Skip CNN stress
- Skip logistic regression
- Use rule-only risk

System still functional.

6.15 Drift Detection

Track:

- Prediction confidence trend
- Regional cluster variance
- Increase in outlier frequency

If anomaly detected:

- Alert admin
- Pause cluster adaptation

6.16 Performance Envelope

Expected load:

- 50–100 images/day
- 10–20 videos/day
- Risk inference per action

Python handles this easily.

For 600 users:

- Scale worker instances
- Possibly move training to separate service

No redesign required.

6.17 Data Privacy Enforcement

Before training:

- Remove user_id
- Remove phone_number
- Remove exact coordinates
- Only keep region-level data

ML training dataset must be anonymized.

6.18 Research Extension Hooks

Later:

- Replace logistic regression with XGBoost
- Add time-series LSTM
- Add advanced crop health segmentation
- Add anomaly detection autoencoder

Because architecture modular, easy to extend.

📘 SECTION 7 — MEDIA PROCESSING IMPLEMENTATION

7.1 Media Architecture Overview

Media pipeline consists of:

1. Frontend upload request
2. Signed URL generation
3. Upload to Cloud Storage
4. MediaUploaded event creation
5. Worker service processes media
6. Extracted features stored in DB
7. MediaAnalyzed event emitted
8. ReplayTriggered (if stress changed)

No synchronous inference in API path.

7.2 Upload Flow (Implementation-Level)

Step 1 — Client requests upload token

API:

POST /api/v1/media/request-upload

Backend:

- Validate JWT

- Generate signed URL (15 min expiry)
- Insert media_files row (status=Pending)

Step 2 — Client uploads directly to Cloud Storage

No file flows through backend memory.

Step 3 — Client confirms upload

POST /api/v1/media/confirm-upload

Backend:

- Verify object exists in bucket
- Publish MediaUploaded event

7.3 Cloud Storage Configuration

Bucket settings:

- Private bucket
- Uniform bucket-level access enabled
- No public ACL
- Lifecycle rule: delete after 90 days
- Versioning optional (off for prototype)
- Object path structure:

/crop_instance_id/media_id/file.ext

Signed URL validity:
15 minutes

7.4 Media Worker Service

Deployed as separate Cloud Run container.

Responsibilities:

- Listen for MediaUploaded events
- Download file from bucket
- Perform inference
- Update media_files.extracted_features
- Emit MediaAnalyzed event

Worker must NOT:

- Modify crop_instances directly
- Trigger replay directly
- Bypass dispatcher

7.5 Image Processing Pipeline

Flow:

1. Load image via OpenCV
2. Compute image_quality_score
    a. brightness check
    b. blur detection (Laplacian variance)
    c. contrast check
3. Reject if quality below threshold
4. Resize to 224x224
5. Normalize pixel values
6. Run PyTorch model inference
7. Compute stress_probability
8. Compute confidence_score

Confidence calculation includes:

- Image quality
- Model output certainty
- Variance across augmentation passes (optional)

Final result stored as:

```
{
  "stress_probability": 0.42,
  "confidence": 0.78,
  "leaf_curl_detected": true,
  "yellowing_score": 0.35
}
```

7.6 Video Processing Pipeline

Video worker steps:

1. Download video
2. Extract frame every N seconds (configurable)
3. Limit max frames (e.g., 30 frames)

4. Run image pipeline on each frame
5. Remove low-quality frames
6. Compute trimmed mean stress
7. Compute variance across frames
8. Reduce confidence if variance high

Final stored:

```
{
  "aggregate_stress": 0.51,
  "confidence": 0.72,
  "frame_count": 18,
  "variance": 0.11
}
```

7.7 Multi-Media Aggregation Logic

If multiple images/videos exist for same day:

Aggregate using:

weighted_mean(stress_i, weight=confidence_i)

If disagreement high:

Reduce final confidence.

No direct stage mutation.

Only stress_score updated.

7.8 MediaAnalyzed Event Handling

Worker publishes:

event_type = "MediaAnalyzed"

Payload includes:

- crop_instance_id
- stress_delta
- confidence_score
- media_id

Dispatcher:

- Inserts event
- ReplayTriggered event created
- CTIS integrates stress in bounded way

7.9 Edge AI Integration (Optional Advanced Mode)

If device uses Edge AI:

- Device computes preliminary stress
- Sends extracted summary
- Backend stores as source='edge'
- Backend still runs verification if full media later uploaded

If low data mode:

Only extracted summary stored.

Confidence lower for edge-only data.

7.10 Retention & Deletion Pipeline

Cloud Scheduler runs daily:

1. Query media_files where scheduled_deletion_at < NOW()
2. Delete object from bucket
3. Keep extracted_features
4. Log deletion in admin_audit_log

Extracted features retained permanently.

Replay unaffected.

7.11 Failure Handling

If worker crashes during processing:

- Event remains Processing
- On restart → reset to Created
- retry_count incremented

- After 5 retries → DeadLetter

DeadLetter visible in admin panel.

7.12 Rate Limiting & Abuse Control

Limits:

- Max images per crop per day (configurable, e.g., 10)
- Max video length (e.g., 60 sec)
- Max video size (e.g., 50MB)

Reject excessive uploads.

Prevents storage abuse.

7.13 Performance Envelope

Expected load:

100–200 users:

- <100 images/day
- <20 videos/day

600 users:

- ~300 images/day
- ~60 videos/day

Scaling strategy:

- Increase worker instances
- Limit frame extraction
- Increase CPU allocation

No redesign required.

7.14 Security Enforcement

Worker:

- Uses service account with bucket access only
- No DB root credentials
- No local file persistence
- Files processed in memory or temp storage

Media never public.

Signed URLs required for access.

7.15 Observability

Worker logs:

- processing_time
- model_version
- confidence_score
- frame_count
- failure_reason

Alerts triggered if:

- processing_time > threshold
- failure_rate spike
- backlog grows

📘 SECTION 8 — API CONTRACT SPECIFICATION

8.1 API Design Principles

1. All endpoints versioned (/api/v1/)
2. All state-changing endpoints require Idempotency-Key header
3. Uniform response envelope
4. JWT required for protected endpoints
5. No direct state mutation without event emission
6. Bulk sync supported for offline mode
7. Long-running tasks tracked via operation_id

8.2 Base URL Structure

/api/v1/

Future breaking changes → /api/v2/

8.3 Standard Response Envelope

Success:

```
{
  "success": true,
  "data": {},
  "meta": {
    "request_id": "uuid",
    "timestamp": "ISO8601",
    "version": "v1"
  }
}
```

Error:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human readable message",
    "details": {}
  }
}
```

8.4 Authentication Endpoints

POST /api/v1/auth/login

Request:

```
{
  "phone_number": "string",
  "password": "string"
}
```

Response:

```
{
  "access_token": "jwt",
  "role": "farmer"
}
```

JWT expiry: 1 hour
Refresh token optional (future)

8.5 Crop Lifecycle APIs (CTIS)

POST /api/v1/crops

Create crop instance.

Headers:
Idempotency-Key: UUID

Request:

```
{
  "crop_type": "Wheat",
  "region": "Haryana",
  "sowing_date": "YYYY-MM-DD"
}
```

Triggers:

- CropCreated event

GET /api/v1/crops

Returns active crops.

Optional:
?include_archived=true

GET /api/v1/crops/{crop_id}

Returns:

- current_stage
- stress_score
- risk_index
- state
- deviation_profile

POST /api/v1/crops/{crop_id}/actions

Headers:
Idempotency-Key: UUID

Request:

```
{
  "action_type": "Irrigation",
  "action_effective_date": "YYYY-MM-DD",
  "local_monotonic_counter": 12,
  "metadata": {}
}
```

Triggers:

- ActionLogged
- ReplayTriggered

POST /api/v1/crops/{crop_id}/sync-batch

Used for offline bulk sync.

Headers:
Idempotency-Key: UUID

Request:

```
{
  "actions": [
    {
      "action_type": "Irrigation",
      "action_effective_date": "YYYY-MM-DD",
      "local_monotonic_counter": 1
    }
```

```
  ],
  "device_session_id": "UUID"
}
```

Server:

- Inserts all valid actions
- Triggers single ReplayTriggered

Response:

```
{
  "accepted_count": 5,
  "rejected": []
}
```

POST /api/v1/crops/{crop_id}/yield

Headers:
Idempotency-Key: UUID

Request:

```
{
  "reported_yield": 4800
}
```

Triggers:

- YieldSubmitted
- ReplayTriggered

8.6 Media APIs

POST /api/v1/media/request-upload

Request:

```
{
  "crop_instance_id": "UUID",
  "file_type": "image"
}
```

Response:

```
{
  "upload_url": "signed_url",
  "media_id": "UUID"
}
```

POST /api/v1/media/confirm-upload

Request:

```
{
  "media_id": "UUID"
}
```

Triggers:

- MediaUploaded

GET /api/v1/media/{media_id}

Returns signed download URL.

8.7 SOE APIs

GET /api/v1/providers

Optional filters:

- equipment_type
- region

Sorted by exposure-adjusted trust.

POST /api/v1/service-requests

Headers:
Idempotency-Key

Request:

```
{
  "crop_instance_id": "UUID",
  "provider_id": "UUID"
}
```

POST /api/v1/service-requests/{id}/complete

POST /api/v1/service-requests/{id}/review

Request:

```
{
  "rating": 4,
  "complaint_category": "Delay",
  "severity_level": 2
}
```

Triggers:

- TrustScoreUpdated

8.8 Admin APIs

Role: admin only

GET /api/v1/admin/dead-letters

POST /api/v1/admin/providers/{id}/verify

POST /api/v1/admin/crops/{id}/force-replay

DELETE /api/v1/admin/users/{id}

Soft delete only.

All actions logged in admin_audit_log.

8.9 What-If Simulation

POST /api/v1/crops/{id}/simulate

Request:

```
{
  "hypothetical_action": {
    "action_type": "DelayIrrigation",
    "days": 3
  }
}
```

Response:

Projected stage, stress, risk.

No event emitted.

8.10 Long-Running Operation Tracking

GET /api/v1/operations/{operation_id}

Returns:

```
{
  "status": "processing",
  "progress": 60
}
```

Used for:

- Video processing
- Replay heavy batches

8.11 Idempotency Rules

All state-changing endpoints require:

Idempotency-Key

Server behavior:

- Store key in idempotency table
- Reject duplicate within window
- Return previous response

8.12 Rate Limiting

Per user:

- 60 requests/min
- Media uploads limited separately
- Admin endpoints stricter logging

8.13 Error Codes (Standardized)

Examples:

- INVALID_INPUT
- UNAUTHORIZED
- FORBIDDEN
- REPLAY_IN_PROGRESS
- DEAD_LETTER_EVENT
- ML_DISABLED
- MEDIA_PROCESSING_FAILED
- RATE_LIMIT_EXCEEDED

8.14 Role Enforcement Matrix

Endpoint	Farmer	Provider	Admin
Create Crop	✓	✕	✓
Log Action	✓	✕	✓

Create Service	✓	✗	✓
Review Service	✓	✗	✓
Dead Letters	✗	✗	✓

Enforced in middleware.

8.15 Health Endpoint

GET /api/v1/health

Returns:

- DB connectivity
- Event queue status
- Worker status
- ML model loaded

Used by Cloud Run health checks.

📘 SECTION 9 — AUTHENTICATION & AUTHORIZATION IMPLEMENTATION DESIGN

This section must align with:

- Phase 3 (Construction) expectations

Lab_Manual

- Your finalized tech stack (JWT-based auth)

CultivaX Tech stack

- Replay integrity (CTIS must be protected)
- Admin audit traceability
- Migration readiness (OAuth/Firebase later)

We will design this cleanly and conservatively.

9.1 Security Design Principles

1. All protected endpoints require authentication.
2. All state mutations require valid JWT + Idempotency-Key.
3. Role-based access control (RBAC) enforced at middleware.
4. Admin actions must be audit-logged.
5. No trust in frontend role flags.
6. JWT must be short-lived.
7. Passwords never stored in plaintext.
8. Soft-delete users instead of hard deletion.

9.2 Authentication Architecture

Authentication Model: Stateless JWT

Flow:

User → Login → Backend validates credentials →
JWT issued →
Frontend sends JWT in Authorization header →
Backend validates token →
Role enforced →
Request processed

No server-side session storage required.

Scalable across Cloud Run instances.

9.3 JWT Structure

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload:

```
{
  "sub": "user_id",
  "role": "farmer",
  "exp": timestamp,
```

```
  "iat": timestamp,
  "iss": "cultivax"
}
```

Important Claims:

- sub → UUID of user
- role → farmer | provider | admin
- exp → 1 hour expiry
- iss → cultivax

Signed using strong secret key (stored in Secret Manager).

9.4 Password Security

Password storage:

- Hashing algorithm: bcrypt
- Salt automatically handled by bcrypt
- Minimum length enforcement
- Optional: password strength validator

Database stores only:
password_hash

Never store raw password.

9.5 Login Flow (Implementation-Level)

1. Receive phone + password
2. Fetch user row
3. Verify bcrypt hash
4. If valid:
    a. Generate JWT
    b. Return token
5. If invalid:
    a. Return UNAUTHORIZED

Rate limit login attempts to prevent brute force.

9.6 Authorization Model (RBAC)

Role matrix:

Endpoint Type	Farmer	Provider	Admin
Crop CRUD	✓	✕	✓
Service listing	✓	✓	✓
Trust update	✕	✕	✓
Dead letter	✕	✕	✓
Force replay	✕	✕	✓

Enforced via FastAPI dependency:

```
def require_role(required_roles):
    ...
```

No business logic inside controller checks.

9.7 Middleware Enforcement Flow

Each protected request:

1. Extract Authorization header
2. Verify JWT signature
3. Validate expiration
4. Extract user_id + role
5. Attach user to request context
6. Apply role guard
7. Continue

If token expired:
Return 401.

9.8 Admin Protection Model

Admin endpoints require:

- role == admin
- additional logging

Before executing admin action:

Insert admin_audit_log entry:

```
{
  admin_id,
  action_type,
  target_id,
  previous_state,
  new_state,
  reason
}
```

No admin action without audit trail.

9.9 Soft Delete Enforcement

When admin deletes user:

UPDATE users SET is_active = false, deleted_at = NOW();

User cannot login afterward.

Historical data preserved.

9.10 Rate Limiting

Per user:

- 60 requests/minute
- 10 media uploads/day
- 5 login attempts/minute

Implemented via:

- Middleware counter (prototype)
- Later: Redis-based limiter

Protects against API abuse.

9.11 Token Refresh Strategy (Phase 1 Conservative)

For prototype:

- Access token: 1 hour
- No refresh token initially

Future:

- Add refresh endpoint
- Store refresh token in DB
- Enable multi-device session control

No redesign needed.

9.12 Device Session Tracking (Offline Mode Support)

For bulk sync:

Include device_session_id in sync requests.

Store:

```
device_sessions:
- session_id
- user_id
- last_seen
- device_info
```

Helps detect:

- Duplicate sync attempts
- Abuse patterns

9.13 Secure Storage on Frontend

Preferred:

- Store JWT in HttpOnly cookie (safer)

Alternative (if needed):

- Store in memory (avoid localStorage)

Never store token in plain localStorage without caution.

9.14 CSRF Protection

If using cookies:

- Enable CSRF token mechanism

If using Authorization header:

- CSRF risk minimal

9.15 HTTPS Enforcement

All endpoints must be served over HTTPS.

Cloud Run enforces TLS by default.

No HTTP allowed.

9.16 Account Lockout Policy

If > 5 failed logins in 10 minutes:

- Temporarily lock account for 15 minutes
- Log event

Prevents brute force.

9.17 Audit Trail Enforcement

All state mutations log:

- user_id
- endpoint
- timestamp
- IP address

Stored in structured logs (Cloud Logging).

9.18 Scaling to 600 Users

JWT-based stateless auth:

- No session storage bottleneck
- No sticky sessions required
- Cloud Run instances can scale freely

No redesign required for 600 users.

9.19 Future Authentication Migration

Because auth logic abstracted:

Later can integrate:

- Firebase Auth
- OAuth
- Aadhaar-like government auth (if required)

Only token verification layer changes.

Business logic unaffected.

9.20 Security Alignment with Lab Manual

Phase 1 requires security requirements in SRS

Lab_Manual

Phase 2 requires architecture + security protocols

Lab_Manual

This section fulfills both:

✓ Encryption method (bcrypt)
✓ Role-based control
✓ Token security
✓ Audit logging
✓ Rate limiting

Fully compliant.

📘 SECTION 10 — FRONTEND INTEGRATION DESIGN

This section must align with:

- Your Canonical Architecture (Time-centric system)

CultivaX

- Lab Phase 2 requirements (System Design + Modeling)

Lab_Manual

- Finalized Tech Stack (Next.js PWA)

CultivaX Tech stack

- Event-driven backend + replay model
- Accessibility-first philosophy
- Offline + sync behavior

This is not UI decoration. This is frontend engineering design.

10.1 Frontend Architecture Overview

Frontend Stack (Locked):

- Next.js (React 18)
- TypeScript (recommended for clarity)
- TailwindCSS
- PWA enabled
- Service Worker for offline caching
- JWT auth via secure cookie

Architecture Pattern:

UI Components
↓
State Layer (Context)
↓
API Client Layer

↓
Backend (FastAPI)

No business logic stored permanently in frontend.

Frontend is presentation + local queue manager only.

10.2 Core Frontend Modules

The frontend is divided into modules aligned with canonical architecture

CultivaX

:

1. Crop Timeline View
2. Action Logging Panel
3. Media Upload Panel
4. What-If Simulator UI
5. Service Marketplace View
6. Dashboard & Insights
7. Admin Panel
8. Accessibility Control Panel

10.3 Crop Timeline UI (Core Experience)

This is the most important screen.

Layout Structure:

Top Section:

- 2D Crop Visualization Canvas
- Weather Indicator
- Risk Indicator

Middle Section:

- Current Day
- Growth Stage
- Stress Indicator Bar
- Drift Indicator

Bottom Section:

- “Recommended Action”
- Primary Button: Mark as Done
- Secondary Button: Override
- “Simulate” Option

Key Rule:
Timeline shifts must reflect backend response, not local assumption.

No optimistic stage mutation without confirmation.

10.4 2D Dynamic Crop Visualization

Implementation Strategy:

- SVG-based dynamic plant rendering
- Growth height mapped to stage progression
- Color hue adjusted via stress_score
- Animated wind effect optional

Mapping Example:

height = stage_progress * max_height
color = healthy_green → yellow → brown

Rendering must be deterministic based on backend data.

No fake visual states.

10.5 Offline Mode Architecture

Frontend must support:

- Action logging while offline
- Media queueing
- Sync on reconnect

Local Storage Strategy:

Use IndexedDB to store:

- Pending actions
- Pending media confirmations
- Device session id
- local_monotonic_counter

Sync Flow:

When connection restored:

1. Batch send /sync-batch
2. Send in chronological order
3. Receive accepted_count
4. Clear confirmed local entries

Important:

Frontend never assumes replay success until server response received.

10.6 Local Monotonic Counter Handling

Each device session:

- Maintains incremental counter
- Attached to each action

This preserves deterministic replay ordering.

If conflict detected:
Server rejects duplicate.

Frontend must handle gracefully.

10.7 Media Upload UI Flow

Steps:

1. User selects image/video
2. Call request-upload endpoint
3. Upload directly to signed URL
4. Call confirm-upload
5. Show “Processing” indicator
6. Poll operation status (if needed)

UI states:

- Uploading
- Processing
- Completed
- Failed (retry available)

Never block UI during inference.

10.8 What-If Simulation UI

Simulation panel:

- Select hypothetical action
- Adjust delay days (slider)
- Click “Simulate”

Frontend calls:

POST /simulate

Display:

- Stage shift
- Stress change
- Risk change

No local calculations allowed.

10.9 Service Marketplace UI

List providers sorted by backend ranking.

Display:

- Trust score
- Completion rate
- Complaint ratio
- Verified badge

Request Flow:

- Select provider
- Confirm request
- Status updates shown in timeline

No direct trust score manipulation on frontend.

10.10 Dashboard & Insights UI

Dashboard displays:

- Crop history
- Deviation trends
- Stress timeline chart
- Yield reports
- Service history

Charts generated using:

- Recharts or Chart.js

All data pulled from backend.

10.11 Accessibility Architecture

Accessibility must be global.

Accessibility Panel Options:

- High Contrast Mode
- Adjustable Text Size
- Reduced Motion Mode
- Brightness Simulation Mode
- Voice Input (optional advanced)
- Language Selection

Implementation:

Global Context:

```
<AccessibilityProvider>
  <App/>
</AccessibilityProvider>
```

CSS variables used to toggle themes.

Must apply across all pages.

10.12 Language Support Strategy

Initial language:

- English
- Hindi (phase 1 optional)

Use i18n framework (next-i18next).

All text externalized.

No hardcoded strings.

10.13 Error Handling UX

Error states:

- API failure
- Replay in progress
- Dead letter state
- Rate limit exceeded

Display friendly messages:

Example:
“Timeline update in progress. Please wait.”

No technical stack traces shown.

10.14 Admin Panel UI

Separate route:

/admin

Features:

- Dead letter viewer
- Force replay
- Provider verification
- User suspension
- ML model toggle
- Event backlog monitor

Admin-only access enforced by role guard.

10.15 PWA Configuration

Enable:

- Service Worker
- Cache static assets
- Cache last known crop state
- Background sync capability

Manifest includes:

- App name
- Icons
- Theme color

Works as installable app.

10.16 Performance Optimization

Techniques:

- Lazy load heavy pages
- Dynamic import for charts
- Compress images before upload (optional)
- Limit re-renders
- Use React.memo for timeline components

10.17 State Management Strategy

Use:

- React Context for global state
- Local component state for UI
- Avoid Redux unless complexity increases

Keep frontend logic simple.

10.18 Security Considerations

- JWT stored in HttpOnly cookie
- No sensitive data in localStorage
- No direct API keys exposed
- Signed URL used for media
- Strict CORS config

10.19 Scalability to 600 Users

Frontend horizontally scalable:

- Static deployment via Vercel or Cloud Run
- No session server required
- Stateless requests

No redesign required.

10.20 Alignment with Lab Manual Phase 2

Lab Phase 2 requires:

- System architecture
- Database schema
- Security protocols

Lab_Manual

This section fulfills UI integration modeling requirement.

📘 SECTION 11 — DEPLOYMENT & INFRASTRUCTURE CONFIGURATION

11.1 Deployment Philosophy

CultivaX deployment must:

1. Be containerized.
2. Be stateless at application layer.
3. Use managed services for persistence.
4. Support horizontal scaling.
5. Separate compute roles (API vs Media Worker).
6. Be environment-separated (Dev / Prod).
7. Be CI/CD ready.

11.2 Infrastructure Overview (GCP)

Final Infrastructure Layout:

Internet
↓
Cloud Load Balancer
↓
Cloud Run (Backend API)
Cloud Run (Media Worker)
Cloud Run (Video Worker - optional)
↓
Cloud SQL (PostgreSQL)
Cloud Storage (Media Bucket)
↓
Cloud Scheduler
Cloud Logging
Cloud Monitoring
Secret Manager

This aligns directly with your locked tech stack

CultivaX Tech stack

.

11.3 Service Breakdown

Service 1 — Backend API

Purpose:

- Authentication
- CTIS
- SOE

- Event dispatcher
- ML inference (lightweight)
- Admin APIs

Cloud Run Configuration:

- CPU: 1 vCPU
- Memory: 512MB (start)
- Max instances: 5 (prototype)
- Concurrency: 80
- Min instances: 0 (cost saving)

Scaling to 600 users:
Increase max instances to 15–20.

Service 2 — Media Worker

Purpose:

- Image processing
- CNN inference
- Feature extraction

Cloud Run Configuration:

- CPU: 1–2 vCPU
- Memory: 1GB
- Concurrency: 1 (important)
- Max instances: 3 (prototype)

Heavy jobs isolated here.

Service 3 — Video Worker (Optional)

Purpose:

- Frame extraction
- Multi-frame aggregation

Higher memory allocation recommended (1.5–2GB).

11.4 Database Deployment

Service: Cloud SQL (PostgreSQL 15)

Configuration:

- Tier: db-f1-micro (prototype)
- Storage: 10GB SSD
- Automatic backups enabled
- Binary logging off (prototype)

Important Settings:

- max_connections tuned
- Connection pooling via SQLAlchemy

Scaling to 600 users:

- Upgrade instance tier
- Add read replica
- Increase storage

No schema redesign required.

11.5 Media Storage Configuration

Service: Cloud Storage

Bucket Settings:

- Private
- Uniform bucket-level access
- Lifecycle rule: delete after 90 days
- Object retention enforced via scheduler

Access:

- Signed URLs (15 min expiry)

No public read access.

11.6 Secrets Management

Use GCP Secret Manager.

Store:

- JWT secret
- DB credentials
- ML model version flag
- Feature flags

Cloud Run service account given access only to required secrets.

Never hardcode secrets.

11.7 Environment Separation

Two environments minimum:

- Development
- Production

Each has:

- Separate Cloud Run services
- Separate Cloud SQL instance
- Separate storage bucket

Environment variables define:

- DATABASE_URL
- ENVIRONMENT
- FEATURE_FLAGS

Prevents accidental data corruption.

11.8 CI/CD Pipeline

Lab Manual Phase 5 requires deployment + optional CI/CD

Lab_Manual

.

Recommended Pipeline:

GitHub → GitHub Actions → Build Docker → Push to Artifact Registry → Deploy to Cloud Run

Pipeline Steps:

1. On push to main:
    a. Run lint
    b. Run tests
    c. Build Docker image
    d. Push image
    e. Deploy to Cloud Run

Frontend pipeline:

- Vercel auto-deploy OR
- Separate Cloud Run service

11.9 Version Control Requirements

All code must be in GitHub (Lab requirement)

Lab_Manual

.

Repository structure:

/backend
/frontend
/infrastructure
/docs

Branches:

- main
- dev
- feature/*

Commit discipline required.

11.10 Logging & Monitoring Integration

Cloud Logging:

Structured JSON logs:

- replay_duration
- event_id
- stress_delta
- trust_score_update

Cloud Monitoring:

Alerts:

- Event backlog > threshold
- Replay failure spike
- Media processing failure spike
- CPU > 80%

Helps during peer testing phase.

11.11 Health Check Configuration

Endpoint:

GET /api/v1/health

Checks:

- DB connection
- Dispatcher operational
- ML model loaded
- Worker connectivity

Cloud Run health check enabled.

11.12 Scaling Plan (100 → 600 Users)

Users	Required Change
100	Default config
200	Increase max instances
400	Increase DB tier
600	Add read replica
1000+	Move to Pub/Sub

Because dispatcher abstraction exists

, migration safe.

11.13 Backup & Disaster Recovery

Cloud SQL:

- Daily automatic backup
- Point-in-time recovery

Cloud Storage:

- 90-day lifecycle
- Optional backup bucket (future)

Code:

- GitHub repository backup

11.14 Security at Infrastructure Level

- HTTPS only
- IAM-based access
- Principle of least privilege
- Service account isolation
- No public DB access
- VPC connector if required

11.15 Cost Control Strategy

Prototype phase:

- Min instances = 0
- Low-tier DB
- Auto scale
- Log retention limited

Avoid over-provisioning.

11.16 Alignment with Lab Manual

Phase 5 requires:

- Deployment
- Hosting
- CI/CD

📘 SECTION 12 — MONITORING, LOGGING & OBSERVABILITY

12.1 Observability Philosophy

CultivaX must be:

- Deterministically debuggable
- Replay traceable
- ML behavior explainable
- Infrastructure observable
- Admin-auditable

Every important state change must be traceable.

No silent failures allowed.

12.2 Structured Logging Strategy

All backend logs must be JSON structured.

Example:

```
{
 "timestamp": "ISO8601",
 "service": "backend-api",
 "event_type": "ReplayTriggered",
 "crop_instance_id": "uuid",
 "actions_processed": 12,
 "replay_duration_ms": 48,
 "stress_delta": 0.04,
 "risk_index": 0.31,
```

```
  "model_version": "risk_v1.0"
}
```

Fields must include:

- request_id
- user_id
- service_name
- severity_level

This enables Cloud Logging filtering.

12.3 Event-Level Traceability

Each event_log entry must include:

- event_id
- event_hash
- status
- retry_count
- processed_at

Replay logs must include:

- snapshot_used
- drift_applied
- stage_transition
- state_before
- state_after

This ensures replay corruption can be audited.

12.4 Request-Level Tracing

Each incoming API request:

- Generate request_id (UUID)
- Attach to context
- Include in all logs

Helps trace:

User action → Event → Replay → State change

Critical for peer testing phase

Lab_Manual

.

12.5 ML Observability

Every ML inference must log:

```
{
  "model_name": "stress_cnn",
  "model_version": "cnn_v1.0",
  "confidence": 0.78,
  "input_quality_score": 0.84,
  "stress_probability": 0.42
}
```

Additionally track:

- Distribution of stress scores
- Drift in prediction averages
- Sudden confidence drops

If distribution shifts unexpectedly → alert.

12.6 Regional Learning Monitoring

Track:

- Number of samples per region
- Cluster size
- Adaptation offset applied
- Variance within cluster

If cluster size < threshold:

Log warning:
"Cluster size insufficient for adaptation."

Prevents silent overfitting.

12.7 Dispatcher Monitoring

Metrics to monitor:

- Pending event count
- DeadLetter count
- Average replay time
- Retry rate
- Processing latency

Alert thresholds:

- 50 pending events
- 5 dead letters in 1 hour
- Replay duration spike > 3x baseline

12.8 Media Worker Monitoring

Track:

- Media processing time
- Failure rate
- Frame extraction count
- Average confidence score

Alert if:

- Processing time > threshold
- Memory spike
- Worker crash loops

12.9 Infrastructure Health Monitoring

Use Cloud Monitoring dashboards for:

- CPU utilization
- Memory utilization
- DB connections
- Cloud Run instance count
- Request latency
- Error rate

Set alerts:

- CPU > 80% sustained
- Error rate > 5%
- DB connection exhaustion

12.10 Replay Debug Mode

Admin-only endpoint:

GET /admin/replay-debug/{crop_id}

Returns:

- Action list
- Snapshot used
- Stage transitions
- Drift adjustments
- Stress timeline

This is extremely powerful during viva.

12.11 Admin Observability Dashboard

Admin UI shows:

- Event backlog
- Dead letters
- ML drift indicator
- Regional cluster stats
- Worker health
- Replay performance graph

All pulled from logs + metrics.

12.12 Log Retention Policy

- Application logs: 30 days
- Error logs: 90 days
- ML training logs: permanent (small)

- Admin audit log: permanent (DB)

Avoid excessive storage costs.

12.13 Incident Response Model

If alert triggered:

1. Admin notified
2. Identify affected crop_instance
3. Inspect event chain
4. Force replay if safe
5. Log resolution

All incidents logged in admin_audit_log.

12.14 ML Drift Detection Strategy

Compute:

Rolling mean of stress_probability

If deviation > threshold:

- Mark ML model unstable
- Disable adaptation
- Alert admin

Never auto-disable silently.

12.15 Abuse Detection Logging

Monitor:

- Excessive override patterns
- Rapid action submissions
- Upload spam
- Repeated login failures

Log with severity="warning".

12.16 Performance Metrics Targets

Target metrics:

- Replay time < 100ms (normal)
- API latency < 300ms
- Media processing < 5 seconds/image
- Video processing < 30 seconds

These targets support scaling to 600 users

CultivaX Tech stack

.

12.17 Alignment with Lab Manual

Phase 4: Peer Testing

Lab_Manual

This section enables structured bug reporting.

Phase 5: Deployment & DevOps

Lab_Manual

Monitoring required.

This section fulfills both.

📘 SECTION 13 — SECURITY IMPLEMENTATION DETAILS

13.1 Security Design Philosophy

CultivaX security model is built around:

1. Integrity of crop timeline (CTIS protection)
2. Identity verification (JWT-based)
3. Event immutability
4. Least-privilege access
5. Media isolation
6. Abuse detection
7. Data minimization
8. Replay protection

Primary threat model:

Not nation-state actors.
But:

- Malicious user manipulation
- Timeline tampering
- Duplicate event injection
- Media abuse
- Trust score fraud
- ML poisoning

13.2 Threat Model Overview

Threat Categories:

1. Authentication attacks
2. Authorization bypass
3. Replay attack (duplicate action injection)
4. Data tampering
5. Media upload abuse
6. Trust score manipulation
7. ML dataset poisoning
8. Infrastructure misconfiguration

Each has mitigation defined below.

13.3 Authentication Security

Mechanisms:

- bcrypt password hashing
- Short-lived JWT (1 hour)
- JWT signature validation
- Account lockout after repeated failures
- Rate limiting on login

Secret key stored in Secret Manager.

Mitigation:

Prevents credential brute force and token forgery.

13.4 Authorization Security

Role-based access control (RBAC).

Enforced in middleware before controller logic.

Admin endpoints double-protected by:

- role check
- audit log requirement

Prevents horizontal privilege escalation.

13.5 Event Replay Protection

Critical area.

Each event:

- Has event_hash (SHA256)
- Unique constraint at DB level
- Idempotency-Key required in API

Mitigation:

- Prevents duplicate action injection
- Prevents network retry duplication

- Prevents offline replay duplication

Additionally:

Chain hash can be added for stronger immutability.

13.6 Input Validation Strategy

All API inputs validated via:

- Pydantic schema
- Type validation
- Range checks
- Enum enforcement
- String length limits

Reject:

- Invalid dates
- Stage skip attempts
- Stress > 1
- Negative land size

Prevents injection and state corruption.

13.7 SQL Injection Protection

- SQLAlchemy ORM
- Parameterized queries
- No raw string interpolation

Prevents SQL injection.

13.8 Media Upload Security

Threats:

- Malware upload
- Oversized file upload
- Storage abuse

- Direct bucket access

Mitigations:

- Signed URLs (15 min)
- Private bucket
- File type validation
- Size limits enforced
- Rate limiting
- No public bucket access

Optional future:

- Virus scanning

13.9 Offline Abuse Detection

Offline mode risk:

- Fabricated chronological manipulation
- Artificial action backdating

Mitigation:

- local_monotonic_counter required
- No backdating beyond tolerance window
- Server-side validation of action_effective_date
- Drift cap enforced

Prevents timeline fraud.

13.10 Trust Score Manipulation Prevention

Threats:

- Fake reviews
- Self-review
- Collusion
- Rating spikes

Mitigations:

- Unique review per request
- Cannot review before completion

- IP anomaly detection
- Sudden rating spike monitoring
- Exposure fairness algorithm

Admin review required for suspicious patterns.

13.11 ML Data Poisoning Mitigation

Threat:

User submits false yield data to manipulate regional learning.

Mitigation:

- Yield_verification_score
- Z-score validation
- Clamp ML_yield_value
- Minimum cluster size threshold
- Manual admin approval for model update

Prevents small-sample corruption.

13.12 Data Anonymization Policy

Before ML training:

Remove:

- user_id
- phone_number
- exact location coordinates
- device identifiers

Keep only:

- region
- crop_type
- anonymized patterns

No personally identifiable data used in training.

13.13 Infrastructure Security

- HTTPS enforced
- No public DB access
- IAM least privilege
- Separate service accounts
- Secret Manager usage
- Environment separation (Dev/Prod)

Prevents configuration-level attacks.

13.14 CSRF & CORS Protection

If JWT via cookie:

- CSRF token required

If JWT via header:

- Strict CORS policy
- Only frontend domain allowed

13.15 Rate Limiting

Limits:

- Login attempts
- Action submissions
- Media uploads
- Simulation requests

Prevents denial-of-service via user abuse.

13.16 Replay Integrity Check

Before replay execution:

- Validate action chronology
- Validate stage consistency

- Validate drift limits

If violation:

- Reject event
- Mark crop_instance RecoveryRequired
- Log incident

Prevents internal corruption.

13.17 Logging Security

Logs must:

- Avoid storing passwords
- Avoid storing raw JWT
- Avoid storing full media content
- Mask sensitive fields

Prevents accidental data leakage.

13.18 Disaster Recovery Security

Backups:

- Cloud SQL automated
- No direct DB shell access
- Controlled restore procedure

Ensures resilience.

13.19 Feature Flags & Kill Switches

Admin can:

- Disable ML
- Disable regional learning
- Disable media processing

Prevents runaway AI behavior.

13.20 Alignment with Lab Manual

Lab Phase 1 requires security in SRS

Lab_Manual

Lab Phase 2 requires security protocols in design

Lab_Manual

This section fully satisfies both.

📘 SECTION 14 — PERFORMANCE & SCALING PLAN

14.1 Performance Philosophy

CultivaX is:

- Logic-heavy
- State-sensitive
- ML-bounded
- Event-driven

The bottleneck is **state integrity**, not request throughput

CultivaX Tech stack

.

We optimize:

1. Replay efficiency
2. DB locking discipline
3. Worker isolation
4. Event backlog control

14.2 User Load Assumptions

Phase 1 (Prototype):

- 100–200 users
- Peak concurrent sessions: 20–30
- Average actions per user per day: 5
- Media uploads per day: 100 images
- Videos per day: 10–20

Phase 2 (Expanded):

- 500–600 users
- Peak concurrent sessions: 80–120
- Media uploads per day: ~300
- Videos per day: ~60

These are realistic agricultural app numbers.

14.3 Replay Complexity Analysis

Replay without snapshot:

O(n) where n = number of actions in crop_instance.

Worst case:

n = 200 actions per crop

With snapshot every 50 actions:

Replay complexity becomes:

O(k), where $k \leq 50$

This guarantees bounded replay time.

Target replay time:

< 100ms (normal)
< 300ms (worst case)

14.4 API Performance Targets

Target metrics:

- Median latency: < 200ms
- 95th percentile latency: < 400ms
- Error rate: < 1%

Non-media endpoints must remain fast.

Heavy tasks offloaded to workers.

14.5 Media Worker Throughput

Image processing:

- Avg time per image: 2–5 seconds
- 100 images/day = trivial load

Video processing:

- Avg time per video: 15–30 seconds
- 20/day manageable with 2 worker instances

Scaling to 600 users:

- Increase worker max instances to 5
- Maintain concurrency=1 per instance

No redesign required.

14.6 Database Load Analysis

Estimated DB operations per action:

- Insert action_log
- Insert event_log
- Replay updates crop_instance
- Possibly snapshot insert

~5–10 DB queries per action

At 600 users:

600 × 5 actions/day = 3000 actions/day

3000 × 10 queries = 30,000 queries/day

This is extremely safe for PostgreSQL Cloud SQL.

Even micro-tier handles this comfortably.

14.7 Lock Contention Analysis

Row-level locking on:

crop_instance_id

Since each farmer operates on separate crop_instance:

No global lock contention.

Worst case:

Same user spams actions → sequential lock.

Safe.

14.8 Event Backlog Thresholds

Safe zone:

Pending events < 20

Warning:

50 events

Critical:

100 events

If backlog > 100:

- Increase Cloud Run max instances
- Reduce replay batch size
- Investigate DB bottleneck

14.9 Scaling Path (100 → 600 Users)

Stage	Required Action
100 users	Default config
200 users	Increase max instances
400 users	Increase DB tier
600 users	Add DB read replica
1000+ users	Migrate dispatcher to Pub/Sub

Dispatcher abstraction ensures migration safe

CultivaX Tech stack

.

14.10 Pub/Sub Migration Trigger Conditions

Move to distributed event bus if:

- Event backlog consistently > 500
- Replay latency > 1 second
- CPU > 80% sustained
- 1000 users

Migration steps:

1. Replace DBEventDispatcher
2. Publish to Pub/Sub
3. Dedicated subscriber service
4. Keep same replay logic

Zero business logic rewrite.

14.11 Horizontal Scaling Strategy

Cloud Run:

- Stateless containers
- Auto-scale on request load
- Independent scaling for:
    - API
    - Media Worker
    - Video Worker

No sticky sessions required.

JWT stateless.

14.12 ML Scaling Strategy

Inference:

Lightweight — stays inside backend.

Training:

Run scheduled batch job.

If dataset grows large:

- Move training to separate container
- Possibly use Cloud Run job

Still no architectural redesign.

14.13 Memory & CPU Planning

Backend API:

- 512MB safe for 200 users
- 1GB for 600 users

Media worker:

- 1GB for image
- 2GB for video

Replay memory footprint small.

No in-memory queue accumulation.

14.14 Cost-Performance Tradeoff

Prototype:

- Min instances = 0

- Small DB tier
- Auto scale

Scaling:

Increase compute only when needed.

Avoid over-provisioning.

14.15 Failure Scenarios & Recovery

Scenario 1: Worker crash

- Event resets to Created
- Retry safe

Scenario 2: API crash

- Stateless → auto restart
- No state loss

Scenario 3: DB overload

- Increase tier
- Add read replica

Scenario 4: ML instability

- Kill switch enabled

System resilient by design.

14.16 Alignment with Lab Manual

Phase 5 requires deployment & hosting

Lab_Manual

.
This section demonstrates performance planning and scalability.

You can defend this mathematically in viva.

📘 SECTION 15 — FUTURE EXPANSION & RESEARCH ROADMAP

15.1 Architectural Evolution Path

Current architecture:

- Monolithic backend
- In-process event dispatcher
- Worker-based media isolation
- Snapshot-optimized replay
- ML embedded inside backend

Future Evolution Path:

Stage 1 → Extract Dispatcher
Stage 2 → Distributed Event Bus (Pub/Sub)
Stage 3 → Microservice separation (CTIS / SOE / ML)
Stage 4 → Multi-region deployment

Because dispatcher abstraction already exists

CultivaX Tech stack

, migration requires config change, not redesign.

15.2 Advanced ML Expansion

Current ML:

- Logistic regression (risk)
- CNN (stress)
- Cluster-based regional learning

Future Enhancements:

1. XGBoost for risk modeling
2. Time-series LSTM for growth forecasting
3. Segmentation-based leaf disease detection
4. Autoencoder-based anomaly detection
5. Bayesian confidence calibration

All possible without breaking CTIS because:

ML influence remains bounded and confidence-weighted.

15.3 Yield Modeling Research Extension

Current:

- Yield stored
- Z-score verification
- ML_yield_value separated from reported_yield

Future:

- Predictive yield modeling
- Early yield forecast at sowing
- Climate-adjusted yield prediction
- Scenario-based probabilistic yield intervals

Important:

Never promise exact yield.
Always probabilistic.

15.4 Regional Intelligence Deepening

Current:

- Regional cluster adaptation
- Minimum sample threshold

Future:

- Sub-district adaptation layers
- Microclimate modeling
- Geo-spatial crop similarity mapping
- Spatial clustering using coordinates (anonymized)

Global model remains stable.
Regional model adaptive.

This preserves research defensibility.

15.5 Edge AI Expansion

Current:

- Backend CNN
- Edge optional (TFLite ready)

Future:

- Fully offline stress detection
- On-device model updates
- Low-bandwidth rural deployment
- Federated learning integration

Federated learning possible because:

- ML separated from CTIS core
- Feature schema versioned

15.6 IoT Integration Roadmap

Future expansion:

- Soil moisture sensors
- Temperature sensors
- NDVI satellite integration
- Automated irrigation triggers

Integration method:

IoT device → API → Event → ReplayTriggered

No structural redesign required.

15.7 Marketplace Expansion

Current SOE:

- Equipment
- Labor
- Inputs

Future:

- Credit scoring integration
- Insurance integration
- Micro-financing
- Cold storage partnerships
- Logistics network

Trust system extensible to:

- Financial reliability scoring
- Service guarantee scoring

15.8 Multi-Tenant SaaS Model

Future:

- District-level deployments
- NGO partnerships
- Government collaboration
- Institutional dashboards

Because system is stateless and cloud-native

CultivaX Tech stack

, multi-tenant extension possible.

15.9 Advanced Visualization Roadmap

Current:

- 2D dynamic SVG crop

Future:

- 3D WebGL-based visualization
- Realistic crop growth modeling
- Augmented Reality field overlay
- Satellite imagery overlay

Must remain optional.

Core logic unaffected.

15.10 Blockchain / Ledger (Deferred Feature)

You intentionally deferred:

Zero-Knowledge Ledger

Future:

- Cryptographic crop history proof
- Immutable yield history
- Bank-readable farm performance proof

Can be implemented as:

Event chain hash → anchored externally.

No core redesign required.

15.11 Distributed Architecture Roadmap

When user base > 1000:

- Replace dispatcher with Pub/Sub
- Separate CTIS service
- Separate SOE service
- Dedicated ML service
- Introduce Redis cache
- Use CDN for media

Because abstraction boundaries already exist

CultivaX Tech stack

, migration safe.

15.12 Research Publication Opportunities

Potential research themes:

1. Time-centric agricultural modeling
2. Human-AI co-pilot systems
3. Drift-bounded adaptive crop modeling
4. Replay-based deterministic agricultural systems

5. Multi-signal stress arbitration
6. Regional adaptive agronomy intelligence

Your architecture supports publishable work.

15.13 Startup Readiness Roadmap

Phase 1: Academic prototype
Phase 2: Pilot in one district
Phase 3: NGO partnership
Phase 4: Marketplace monetization
Phase 5: SaaS model

Revenue models possible:

- Commission on services
- Premium analytics
- Subscription advisory
- Institutional dashboards

Architecture supports monetization.

15.14 Compliance & Policy Expansion

Future:

- Data governance compliance
- Government integration
- Agricultural data privacy compliance
- Digital agriculture mission alignment

Because system logs structured and anonymizes ML data, compliance feasible.

15.15 Final Architectural Maturity Summary

CultivaX now:

✓ Deterministic replay core
✓ Event-driven architecture
✓ ML bounded and versioned

✓ Worker-isolated media
✓ Marketplace fairness
✓ Cloud-native deployment
✓ Migration-ready abstraction
✓ Offline-safe design
✓ Accessibility-ready frontend
✓ Lab-compliant lifecycle alignment